

Remember: you may work in groups, but must write up your solution entirely on your own. Solutions must be typeset (L^AT_EX preferred but not required). Collaboration is limited to discussing the problems – you may not look at, compare, reuse, etc. any text from anyone else in the class. You may use the internet to look up formulas, definitions, etc., but may not simply look up the answers online.

Please include proofs with all of your answers, unless stated otherwise.

1 Amortized analysis of 2-3-4 trees (33 points)

Recall that in a 2-3-4 tree, whenever we insert a new key we immediately split (on our way down the tree) any node we see that is full (has 3 keys in it).

- (a) (11 points) Show that in the worst case, the number of splits that we do in a single operation can be $\Omega(\log n)$. In other words, show that there is a series of n inserts so that the next insert causes $\Omega(\log n)$ splits. You can be informal here — just explain at a high level what such a sequence would look like and why it would result in $\Omega(\log n)$ splits.

Let's try to get around this worst-case bound by using amortized analysis. We will try to prove that the amortized number of splits is only $O(1)$. Note that this does not mean that the amortized running time of an insert is $O(1)$ (since this is not true); it just means that the amortized number of splits is $O(1)$. So think of “cost” not as “time”, but as “number of splits”. Since we didn't talk about them in class, feel free to assume there are no delete operations.

- (b) (11 points) Since we only have to split a node when it's full, a natural approach is to have a bank on every node which works as follows. When a node is first created its bank is initialized to 0. When a node becomes full, we add 1 to its bank. Then when we split a node we use the 1 in its bank to pay for the split. In other words, a node's bank is 1 if the node is full and 0 otherwise.

This argument unfortunately does not work. Explain why this banking scheme *does not* imply that the amortized number of splits is $O(1)$. Your explanation should be convincing, but does not need to be a fully formal proof

- (c) (11 points) Show how to modify this argument to work: use a bank at each node to prove that the amortized number of splits is $O(1)$. Hint: the previous part shows that this bank cannot just equal 1 if the node is full and 0 otherwise.

2 Move to Front (34 points)

Amortized analysis can be used not just to give absolute bounds on running times, but also bounds *compared to other algorithms*. In this problem we'll see an example of this.

Suppose we have n items $x_1, x_2, \dots, x_n$ that we wish to store in a linked list. The cost of a *lookup* operation is the position in the list, i.e. if we lookup some element x_i we pay 1 if it is the

first element, 2 if it is the second element, 3 if it is the third element, etc. This models the time it takes to scan through the list to find the element.

For example, suppose there are four items and we lookup x_1 twice, x_2 three times, x_3 once, and x_4 four times. Then if we store them in the list (x_1, x_2, x_3, x_4) the total cost is $1(2) + 2(3) + 3(1) + 4(4) = 27$. On the other hand, if we stored them in the list (x_4, x_2, x_1, x_3) the cost would be $1(4) + 2(3) + 3(2) + 4(1) = 20$. It is easy to see that if we knew the number of times each element was looked up, the optimal list is simply sorting by the number of lookups, i.e. the first element is the one looked up most often, then the element looked up next most often, etc.

But what if we do not know how many times each element will be looked up? It turns out that a good strategy is *Move-to-Front* (MTF): when we lookup an item, we also move it to the head of the list. Let's say that moving an element is free (it is easy enough to implement with a constant number of pointer switches, in any case). So if, for example, we start with the list $(x_1, x_2, \dots, x_n)$ and then lookup x_4 , we pay a cost of 4 and afterwards the list will be $(x_4, x_1, x_2, x_3, x_5, x_6, \dots, x_n)$. Then if we lookup x_4 again we only have to pay a cost of 1.

- (a) (17 points) Consider a sequence of m operations (think of m as being much larger than n). Let C_{init} be the total cost of these operations if we used the original list $(x_1, x_2, \dots, x_n)$ the entire time (i.e., we do not use MTF). Let C_{MTF} be the total cost if we use MTF on the same operations starting from the original list. Prove that $C_{MTF} \leq 2C_{init}$.

Hint: Think of each item x_i as having a “piggy bank” in which the number of tokens is the number of elements before it in the current list that come after it in the initial list, or a potential function equal to the number of reversals.

- (b) (17 points) Now let C_{static} be the smallest possible cost among all fixed lists. In other words, given the same sequence of lookups, each fixed list (without MTF) has some cost, and let C_{static} be the minimum of those costs. Note that this will correspond to the list that is sorted by the number of accesses, like in the first example. Prove that $C_{MTF} \leq 2C_{static} + n^2$.

Hint: Think about your bank or potential function argument from the previous part. What if you redefine it to be relative to the optimal list, rather than the initial list? Then what is the initial bank/potential value? How does this affect the amortized analysis?

3 k -th smallest elements (33 points)

Given an array $a_1, a_2, \dots, a_n$ and an integer $k \in [n]$. For each $i \in [n + 1 - k]$, let b_i be the k -th smallest element in $a_1, \dots, a_{k+i-1}$.

- (a) (17 points) Design an algorithm which outputs the array $b_1, \dots, b_{n+1-k}$. The running time should be $O(n \log k)$.
- (b) (16 points) Prove the correctness and the running time for your algorithm.